

Тестирање на апликацијата **FencingCoach**

Документација за систематско тестирање

Предмет:

Софтверски квалитет и тестирање

Изработил:

Петар Србиноски 231139

Линк до репозиториум: <https://github.com/petarsrbinoski/fencingcoach>

Скопје, септември 2026

Документација за тестирање на FencingCoach

Документот објаснува зошто се избрани тестовите, како се извршени и што покажале. Покрај проверките на правилата и корисничките текови, содржи и посебна проверка на одговорите од Gemma E4B, модел што работи локално на лаптопот. Резултатите се претставени со нивните ограничувања, бидејќи еден успешен одговор не значи дека моделот секогаш ќе одговори исто.

Содржина

Поглавје	Содржина
1	Вовед, цел и опфат
2	Стратегија за избор на тестови и влезни блокови
3	Независен почетен сет и покриеност
4	Тестирање базирано на својства
5	Мутациско тестирање
6	PostgreSQL и HTTP API тестови
7	End-to-end тестови со Playwright
8	Автоматизација и повторување
9	Проверка на одговорите од локалниот модел
10	Резултати, ограничувања и заклучок

1 Вовед, цел и опфат

FencingCoach е апликација за еден корисник со Next.js интерфејс, FastAPI сервер, PostgreSQL база и интеграции со Garmin и јазичен модел. Функциите што се тестираат опфаќаат неделен распоред, извлекување метрики, пресметка на нутритивни цели, промена на типот на ден и евиденција на оброци. Целта е да се утврди дали независно изработени примери, генерирани својства и регресиски тестови разликуваат реални промени во однесувањето.

1.1 Граници на проверката

Постоечките `backend/tests/` се регресиски сет на проектот и не влегуваат во независните мерења на покриеност и мутации. За тие мерења се користат одделни тестови со синтетички податоци, мигрирана PostgreSQL база и вистински HTTP и Chromium текови. Во новата проверка на одговори, наместо замена за јазичниот модел, се повикува Gemma E4B преку јавниот интерфејс на coach агентот. Не се користат лични податоци.

Ниво / техника	Што докажува	Алатка / граница
Примери и влезни блокови	Очекуван исход за валидни, невалидни и гранични вредности	pytest; јавни service функции
Својства	Инваријанти низ семејства на влезови	Hypothesis; чисти функции и изолирана БД
Мутации	Дали избраниот сет открива намерно изменет код	mutmut; само распоред и Garmin
Интеграција и API	Миграции, HTTP одговори и состојба	FastAPI + PostgreSQL 16
End-to-end	Видливо однесување низ вистински интерфејс	Playwright + Chromium
Одговори од локален модел	Дали coach агентот користи вистински податоци, признава празнини и прави само побарани промени	Gemma E4B преку llama_server; по еден повик во даден момент

Овие нивоа беа избрани затоа што секое открива различен вид проблем. Малите примери кажуваат дали познато правило дава точен резултат; тестовите со генерирани влезови проверуваат дали правилото важи и надвор од неколку рачно избрани примери. API и browser тестовите проверуваат дали тие правила навистина стигнуваат до корисникот и дали промените остануваат зачувани. Ниту едно ниво самостојно не би дало целосна слика.

2 Стратегија за избор на тестови и влезни блокови

За почетниот сет е применето функционално партиционирање: секој договор е поделен на блокови со различно очекувано однесување, а потоа се избрани претставници и соседни вредности кај границите. Ова не е формално Base Choice Coverage за еден метод, методот е сличен по систематскиот избор на блокови, но нема такво тврдење за покриеност.

2.1 Карактеристики и блокови

Област	Основен валиден блок	Варијанти и граници
Неделен распоред	Седум препознаени вредности, понеделник до недела	0, 6 и 8 записи; непознат или празен токен; празни места и големи букви
Garmin скалар	Присутна вредност во дозволен опсег	Отсутна, null, ненумеричка, под/на самата/над границата; HRV = 0
Garmin приоритет	Вредност на првата поддржана патека	Прва патека недостасува; резервна патека; присутна, но невалидна прва вредност
Нутритивна цел	Валиден профил, фаза и тип на ден	Нема профил -> 89 kg; 60/80 kg; различна фаза и тип на ден
Историја на калории	Пет употребливи дена во претходните 14	Четири дена; ден -14, ден -15 и денес; невалиден статус
Рачен тип на ден	Запис за конкретниот датум	Замена, бришење, автоматски избор и соседен датум

2.2 Изведени случаи и очекувања

Случај	Избран влез	Очекуван резултат
S1	Седум валидни типови со мешана големина на букви	Нормализација без промена на редоследот
S2	Шест или осум типови; непознат токен	Одбивање пред употреба на распоредот
G1	HRV 0, 1 и 300	Нулата е недозволена; двете граници се валидни
G2	Нема прва HRV вредност, постои неделен просек	Резервната патека дава употреблива вредност
N1	Четири наспроти пет употребливи денови	Формула наспроти измерена основа за одржување
N2	Запис на ден -14, -15 и денес	Само претходните 14 дена учествуваат
D1	Рачна промена на еден датум	Промената е трајна и не го менува соседниот датум

Граничните вредности се важни бидејќи таму најчесто се гледаат грешки што не се забележуваат со обичен, „среден“ пример. HRV од нула мора да се одбие, додека 1 и 300 се дозволени граници; ако тестиравме само 50, таква грешка лесно ќе поминеше. Кај историјата на калории, четири и пет дена ја раздвојуваат гранката што користи формула од гранката што користи измерени податоци. За рачниот тип на ден проверуваме и соседен датум, затоа што грешка таму би променила повеќе од еден ден иако корисникот уредил само еден.

3 Независен почетен сет и покриеност

Почетниот сет е замрзнат пред додавање на својства и мутациски регресији. Трите датотеки во `testing/baseline/` ги проверуваат распоредот, Garmin извлекувањето и нутритивните цели. Нутритивните случаи работат со празна мигрирана база и синтетички спортист. Очекуваните исходи се запишани пред извршување на мутациите.

Мерен модул	Покриени / вкупно гранки	Примери што придонесуваат
Распоред	4 / 4	Должина, валидност и редослед
Garmin извлекување	22 / 32	Статуси, граници, патеки и payload
Нутритивни цели	17 / 24	Историја, профил и тип на ден
Вкупно	43 / 60 (71,7%)	24 успешно поминати pytest случаи

Почетниот сет е намерно одделен од веќе постоечките тестови. Така можам да видам што откриваат само новоизбраните примери, пред подоцна да додадам генерирани случаи и регресији. Покриеноста 43/60 значи дека тие примери поминале низ 43 од 60 измерени гранки во три избрани модули. Тоа не значи 71,7% од целата апликација, ниту докажува дека посетените гранки се без грешки.

4 Тестирање базирано на својства

Hypothesis генерира семејства на поддржани влезови, а тестовите проверуваат правила што треба да важат за секој пример. Почетните датотеки не се менувани. Додадени се 13 функции: четири за распоред, шест за Garmin и три со PostgreSQL за нутритивните цели. Поставката `derandomize=True` ја прави селекцијата повторлива; бројот на генерирани примери е различен од бројот на pytest функции.

Област	Генератор / инваријанта	Поставени примери
Распоред	Валидни низи, нормализација, редослед и одбивање на невалидни токени	4 × 100
Garmin	Скаларни граници; речници со изоставени и null полиња; ограничени листи за батерија, readiness и VO ₂ max	6 × 100
Нутритивни цели	Тежина и тип на ден, енергетска согласност, изолирани датуми и историја	20 + 15 + 15
Вкупно поставени	13 property функции	1 050

За Garmin, секоја метрика со статус `ok` мора да има конечна вредност во документируваниот опсег; другите статуси немаат доверлива скаларна вредност. Генераторите вклучуваат дозволени нумерички стрингови и облици на листи што ги прифаќаат `extractors`. `NaN` и бесконечност се посебни `in-process robustness` случаи, а не JSON податоци. За нутритивните цели се проверува согласноста меѓу `kcal` и макронутриентите и правилата за профил, датум и одржување.

Секој генериран PostgreSQL пример повикува заштитено ресетирање и повторно внесување на синтетичките редови во сопственото тело. Само `function-scoped fixture` не би ја изолирала базата меѓу примерите што Hypothesis ги извршува во една pytest функција. Проширениот запис покажува 37/37 успешно поминати случаи и 53/60 гранки (88,3%) за истиот измерен `production` код. Нема најдена грешка при генерирањето, па нема измислен минимизиран контрапример.

Hypothesis е избран за правила што треба да важат за многу облици на влез, а не само за една конкретна вредност. На пример, за Garmin податоците комбинираме присутни, празни и невалидни полиња и проверуваме дека статусот „`ok`“ никогаш не носи невозможна вредност. За нутритивните цели проверуваме согласност меѓу

енергијата и макронутриентите низ различни тежини и денови. Генераторот не докажува дека сме ги испробале сите влезови, но е добра проверка на правилото против варијанти што рачно можеби би ги пропуштиле.

5 Мутациско тестирање

Mutmut 3.8.0 создава изменети верзии на кодот и ги извршува избраните тестови. За споредливост се мутираат само `schedule.py` и `garmin_extract.py`; нутритивните, API, browser и постоечките регресиски тестови не учествуваат во резултатот. Трите фази користат свежи работни простори, ист hash на двата production фајла и исти 466 mutant ID. Пред мерењето биле проверени чистото извршување и способноста на инструментот да убие намерно избран mutant.

Фаза	Избрани тестови	Убиени	Преживеани	Резултат
A	Почетни примери за распоред и Garmin	248	218	53,2%
B	A + својства за тие два модула	265	201	56,9%
C	B + три Garmin регресији	287	179	61,6%

Нема mutant без тест, timeout, грешка или прескокнат mutant. Резултатот е убиени / (убиени + преживеани + без тест) и нема корекција за еквивалентност. Во B, генерираните облици откриваат уште 17 промени, главно во калории, body battery, training readiness и VO₂max. Во C, три случаи откриваат дополнителни 22: предност на вгнездена sleep патека, предност на директна body-battery вредност и задржување на дијагностички payload.

На пример, mutant што ја отстранува патеката `dailySleepDTO.sleepTimeSeconds` менува резултат од 2 на 1 час кога таа содржи 7 200 секунди, а резервната патека 3 600. Регресискиот тест ја прави разликата видлива. Друг mutant менува само формулација на детал за празен status payload; не е додаден кршлив тест за буквален текст без договор за него. Сите преживеани остануваат во именителот. Целосната анализа и зачуваните A, B и C резултати го ограничуваат толкувањето на двете мутирани датотеки.

Мутациското тестирање одговара на друго прашање: дали тестовите би забележале намерно погрешна промена во кодот? Ако мутација преживее, тоа значи дека избраниот сет не ја разликува од оригиналот; не значи автоматски дека апликацијата има реален дефект. Затоа ги споредив истите 466 мутации во три фази. Растот од 248 на 287 откриени мутации покажува дека генерираните случаи и насочените регресији навистина додаваат способност за откривање, а не само повеќе тестови на хартија.

6 PostgreSQL и HTTP API тестови

Посебен Docker Compose проект создава празна PostgreSQL 16 база, ги применува Alembic миграциите и стартува вистински FastAPI сервер. Тестот повикува HTTP endpoint, а потоа ја проверува и зачуваната состојба. Секој случај ги ресетира само синтетичките редови во тест-базата; заштитниот механизам ја проверува дестинацијата пред празнењето. Асинхроното генерирање оброк се набљудува со ограничено polling додека job не стигне во терминална состојба.

Сценарио	Главни проверки
Почетен smoke	Здравје на БД, денешни цели, завршен синтетички estimate job, без зачуван оброк
Рачен тип на ден	Заменува ист датум, одбива невалиден тип, не менува соседен датум, брише во Auto
Оброк	Estimate не логира; прегледаните вредности се менуваат, логираат еднаш, се сумираат и бришат
Неуспех и закрепнување	Контролиран estimate error е терминален; непознат ID дава 404; следна процена успева
Workout override	Замена, читање, валидација на sets/ reps/load/RPE, независен датум и бришење

Сите пет API/database случаи поминаа. Тест-околината го заменува само надворешниот нутритивен агент и USDA збогатувањето со предвидливи резултати; routing, background job, business logic и PostgreSQL се реални. JUnit и метаподатоците го идентификуваат извршувањето.

Овие тестови се избрани за грешки што чиста функција не може да ги открие: миграција што не се применува, HTTP одговор што не одговара на договорот или податок што не останува зачуван. Проверката на оброкот намерно ги разделува проценката и логирањето, бидејќи корисникот мора да може да ги прегледа и поправи бројките пред да ги зачува. Тука нутритивниот модел е заменет со предвидлив одговор, за евентуален пад да се припише на API и базата, а не на надворешна услуга.

7 End-to-end тестови со Playwright

Playwright ја отвора production-изградбата на Next.js во Chromium и работи преку видливите контроли. Секој случај добива нов browser context, нова синтетичка состојба во БД и фиксирано пладне UTC за истиот ден што го користи backend. Селекторите се засновани на улоги и достапни имиња.

Бр.	Корисничко сценарио	Потврден исход
1	Страницата за исхрана се отвора и бара estimate	Вистински цели; контролирана процена стигнува до преглед
2	Корисник менува тип на ден и повторно вчитува	Рачниот избор и целите опстојуваат; Auto го враќа автоматскиот избор
3	Корисник прегледува, менува и зачувува оброк	Еден оброк и точни суми по reload; бришењето опстојува
4	Estimate паѓа, а потоа корисник се обидува повторно	Копчето се ослободува; нема оброк од грешката; нов преглед успева

Сите четири Chromium случаи поминаа без retry. Browser тестот за оброк ги проверува и прикажаните суми по повторно вчитување и HTTP агрегатите, наместо да се потпира само на појавување на текстот на оброкот. Playwright JSON е зачуван одделно од API JUnit.

Browser случаите ги избрав според задачите што корисникот навистина ги извршува: отворање исхрана, промена на ден, зачувување оброк и опоравување по грешка. Тие проверуваат дека копчињата, пораките и податоците по повторно вчитување се усогласени со серверот. Пад тука може да открие проблем во поврзувањето меѓу frontend и backend што API тест сам по себе не го гледа. Четири сценарија ги покриваат главните текови за исхрана, но не секоја страница на апликацијата.

8 Автоматизација и повторување

Извршувањето е преку локални bash скрипти, не преку CI. Скриптите создаваат единствен Compose проект и волумен, применуваат миграции, собираат JUnit/coverage/Playwright докази и ја отстрануваат само својата тест-

околина. Изворните и тест-датотеките се идентификуваат со hash; mutation фазите зачувуваат конфигурација, mutant ID и генериран код. Независниот сет не го чита главниот .env; живите LLM проверки користат посебни привремени променливи за поврзување со локалниот модел.

```
uv sync --frozen
bash testing/scripts/run_backend_tests.sh baseline
bash testing/scripts/run_backend_tests.sh expanded
bash testing/scripts/run_e2e.sh
bash testing/scripts/run_mutation_stage.sh A
bash testing/scripts/run_mutation_stage.sh B
bash testing/scripts/run_mutation_stage.sh C
```

Пред browser извршување се потребни npm ci и npx playwright install chromium во frontend/, како и Docker Compose. Командите за mutation се извршуваат последователно за споредливо времетраење. Тестирачкиот прирачник ги наведува портите и променливите за околината. Репозиториските quality gates се одделни од независните мерења.

Автоматизацијата не е само за побрзо повторување. Со исти команди и изолирана тест-база може повторно да се провери дали некоја промена го расипала претходно потврдено однесување. Скриптите ги пуштаат зависните фази по ред и ги чуваат резултатите, па споредбата меѓу фазите има смисла. Живите LLM случаи исто така се пуштаат еден по еден, бидејќи локалниот сервер има само едно место за барање.

9 Проверка на одговорите од локалниот модел

Досегашните тестови покажуваат дека апликацијата ги подготвува барањата, ги обработува грешките и го зачувува резултатот. Кога во тестот користиме замена за моделот, сепак не дознаваме што навистина ќе му каже на спортистот. Затоа додадовме пет кратки проверки со Gemma E4B, кој работи на лаптоп преку llama_server на порта 8080. Секој тест му дава измислен контекст на истиот coach агент што го користи апликацијата и го оценува видливиот одговор.

Ги избравме случаите според можните последици за корисникот. Измислена HRV вредност би можела да направи советот да изгледа лично заснован кога не е. Пропуштена вредност за сон покажува дека моделот може да даде општ совет без јасно да каже на кој податок се потпира. Погрешно додаден натпревар, пак, ја менува состојбата на календарот. Затоа ги проверуваме и обичните одговори и двете страни на дејството: кога недостасува датум, треба да праша; кога датумот и приоритетот се дадени, треба да зачува точно еден запис.

Случај	Зошто го избравме	Што точно проверува
HRV е познат	Проверува дали моделот чита вистински податок наместо да претпоставува.	Со дадени 55 ms, одговорот мора да ја содржи вредноста 55; календарот и тренингот остануваат исти.
HRV недостасува	Проверува дали моделот умее да признае дека нема податок.	Не смее да наведе измислен број за HRV; не смее да направи промена во базата.
Совет по пет часа сон	Проверува дали советот се врзува за конкретната причина за заморот.	Одговорот треба да ги спомне петте часа сон; прашање за совет не смее да го измени тренингот.
Натпревар без датум	Недоволен влез не треба да стане вистински запис со измислен датум.	Агентот бара датум и не додава натпревар.
Натпревар со датум	Проверува дали изрично побарана промена навистина се случува еднаш.	Преку API се чита еден запис: Harbor Open, 14.03.2027, приоритет B.

Тестовите се во backend/tests/test_live_llm_responses.py. Се вклучуваат само кога RUN_LIVE_LLM_EVALS=1, па во вообичаеното pytest извршување се прескокнуваат. llama_server има само едно слободно место за барање, затоа

петте случаи се пуштаат последователно, без паралелни pytest работници. Во тестовите се користи привремена SQLite база и измислени податоци; не се менува вистинскиот календар.

9.1 Забележани одговори и граници

Целосното последователно извршување даде 4 успешни од 5 случаи. Gemma E4B правилно ги повтори 55 ms HRV, кажа дека непостоечка HRV вредност не е достапна, побара датум пред да додаде натпревар и зачува еден натпревар со побараните датум и приоритет. Кај советот по пет часа сон, моделот предложи полесен тренинг, но во тој обид не ги спомна петте часа. Истиот случај успеа во уште два одделни обиди, па неуспехот покажува променливост во формулацијата, а не постојано погрешен совет.

Ова е мал примерок од однесувањето на моделот. Тестовите не докажуваат медицинска точност, нутритивна точност или добар совет за секоја можна состојба. Не бараат идентична реченица, се проверуваат точните лични податоци, отсуството на измислена HRV бројка, отсуството на внатрешен текст од размислување и видливиот резултат во календарот. За посилна оценка треба истите сценарија да се повторуваат повеќе пати и човек да ги прегледа советите.

10 Резултати, ограничувања и заклучок

Проверка	Потврден резултат
Независен почетен сет	24/24 успешни; 43/60 гранки во избраните service датотеки
Проширен сет со својства	37/37 успешни; 1 050 генерирани примери; 53/60 гранки
Мутациско тестирање	466 исти мутации; во фазите A, B и C се откриени 248, 265 и 287
PostgreSQL и HTTP API	5/5 успешни
Playwright и Chromium	4/4 успешни
Локален Gemma E4B	4/5 во целосно извршување; случајот за сон успеа во 2 од 3 обиди
Репозиториски pytest без жив модел	248 успешни; 5 LLM проверки прескокнати

Овие резултати даваат доверба во избраните правила, зачувувањето на податоците и неколку важни кориснички текови. Живиот локален модел беше проверен со синтетички примери, но тоа не ја опфаќа целата негова променливост. Не се тестирани реална Garmin синхронизација, перформанси и медицинска точност.